



Proyecto 1: Planificación, Análisis y Diseño de Pruebas

CS5383 · Verificación y Pruebas de Software

1. Contexto del proyecto

Este proyecto extiende lo trabajado en el Laboratorio 1. Cada grupo recibe un **caso de negocio** construido específicamente alrededor de un **sistema real y funcional**, sobre el cual deberá desarrollar las primeras actividades del proceso de pruebas según ISTQB:

1. **Planificación** — definir el enfoque, la estrategia y el plan de pruebas.
2. **Análisis** — analizar la base de pruebas y derivar condiciones de prueba.
3. **Diseño** — construir casos de prueba a partir del análisis de pruebas
4. **Ejecución manual acotada** — ejecutar a mano los casos de mayor prioridad y reportar los hallazgos más relevantes.

2. Objetivos de aprendizaje

- Aplicar el proceso de planificación de pruebas a un caso con contexto de negocio realista.
- Analizar una base de pruebas (requisitos derivados del comportamiento real del sistema) para identificar condiciones de prueba trazables.
- Diseñar casos de prueba aplicando las técnicas de diseño, justificando su elección.
- Practicar la priorización de esfuerzo de pruebas con base en el riesgo del producto.

- Ejecutar manualmente los casos de mayor prioridad y reportar hallazgos, generando criterio propio (basado en evidencia, no en suposición) para decisiones futuras de automatización.

3. Formación de grupos y asignación de casos

- Grupos de **máximo 2 integrantes**.
- **Cada grupo elige 1 de los 5 casos**. Si se forman más de 5 grupos, se puede repetir un caso — en ese escenario, ambos grupos trabajan el mismo sistema pero deben desarrollar su plan, análisis y diseño de forma independiente (no se comparte trabajo entre ellos).
- El caso elegido se usará también en el Proyecto 2 — elíjanlo pensando en eso.

4. Casos de proyecto

Cada caso incluye una empresa y problemática de negocio, el sistema real donde se sustenta esa operación, su alcance funcional, requisitos funcionales y no funcionales, restricciones y riesgos iniciales. **Los riesgos iniciales no son exhaustivos** — parte del trabajo de análisis es que el grupo identifique riesgos adicionales.

Caso 1 — Banca digital (ParaBank)

Sistema real: ParaBank — <https://parabank.parasoft.com/parabank/index.htm>

Empresa y problemática: ParaBank es un banco digital que ofrece cuentas corrientes y de ahorro, transferencias entre cuentas, pago de servicios, solicitud de préstamos y atención al cliente, todo desde su portal en línea. El banco está por lanzar una campaña de captación masiva de nuevos clientes y necesita asegurar que los flujos financieros críticos —apertura de cuentas, transferencias y pago de servicios— funcionen sin errores antes del lanzamiento, dado que cualquier falla en el manejo de dinero de un cliente tiene un impacto reputacional y regulatorio severo.

Alcance funcional del sistema: registro y login de clientes, apertura de nuevas cuentas (corriente/ahorro), transferencia de fondos entre cuentas propias, pago de servicios a terceros (bill pay), búsqueda de transacciones, actualización de datos de contacto, solicitud de préstamos, panel de administración para configurar parámetros del banco.

Requisitos funcionales clave:

1. El registro debe exigir todos los campos obligatorios y validar que el nombre de usuario no esté ya en uso.
2. El login debe rechazar credenciales inválidas con un mensaje de error claro.
3. La apertura de una nueva cuenta debe requerir seleccionar el tipo de cuenta y una cuenta de fondeo existente.
4. Una transferencia de fondos debe debitar la cuenta origen y acreditar la cuenta destino por el mismo monto.
5. El pago de servicios (bill pay) debe requerir nombre del beneficiario, dirección, datos de cuenta y monto, y debe descontar el monto de la cuenta seleccionada.
6. La búsqueda de transacciones debe soportar búsqueda por fecha, rango de fechas, monto o ID de transacción.
7. La solicitud de préstamo debe evaluar el monto solicitado contra el monto de enganche (down payment) y devolver una aprobación o rechazo.
8. La actualización de datos de contacto debe persistir los cambios y reflejarlos en el perfil del cliente.

Requisitos no funcionales: disponibilidad alta durante horario bancario, tiempo de respuesta menor a 3 segundos por transacción, manejo consistente de sesión (cierre automático por inactividad).

Riesgos iniciales: transferencias que no cuadran (débito sin crédito o viceversa), préstamos aprobados incorrectamente, pérdida de sesión durante una transacción en curso.

Caso 2 — E-commerce (Advantage Online Shopping)

Sistema real: Advantage Online Shopping — <https://www.advantageonlineshopping.com/>

Empresa y problemática: Advantage es una tienda en línea que vende accesorios y productos de marca propia, con carrito de compras, tarjetas de regalo y cuentas de usuario. La empresa está a semanas de su campaña de temporada alta y le preocupa que errores en el carrito de compras o en la aplicación de tarjetas de regalo generen pérdida de ventas o descuentos mal calculados, afectando directamente el margen del negocio.

Alcance funcional del sistema: registro y login de usuarios, navegación y búsqueda de productos por categoría, carrito de compras, aplicación de tarjetas de regalo/promociones, checkout con dirección de envío y método de pago, lista de deseos (wishlist), historial de pedidos.

Requisitos funcionales clave:

1. El registro y login deben validar los campos obligatorios y mostrar errores claros ante datos inválidos.
2. La búsqueda de productos debe devolver resultados relevantes al término buscado.
3. Agregar un producto al carrito debe actualizar el contador de items y el subtotal correctamente.
4. Quitar un producto del carrito debe recalcular el subtotal correctamente.
5. Aplicar una tarjeta de regalo o código promocional debe reducir el total en el monto correcto, sin permitir montos negativos.
6. El checkout debe exigir dirección de envío y método de pago antes de confirmar la compra.
7. Al confirmar una compra, el pedido debe aparecer correctamente en el historial de pedidos del usuario.
8. Agregar un producto a la lista de deseos no debe afectar el carrito de compras ni generar un cargo.

Requisitos no funcionales: el carrito debe mantenerse consistente si el usuario navega entre páginas, tiempo de carga de catálogo menor a 2 segundos, cálculos monetarios sin errores de redondeo.

Riesgos iniciales: descuentos mal calculados, pérdida del contenido del carrito al navegar, pedidos duplicados por doble clic en "Comprar".

Caso 3 — E-commerce con panel administrativo (OpenCart)

Sistema real: OpenCart Demo — <https://demo.opencart.com/>

Empresa y problemática: Esta tienda opera sobre OpenCart, una plataforma de e-commerce con catálogo multi-categoría, cupones de descuento y un panel de administración donde el equipo de operaciones gestiona productos, stock y pedidos. La empresa ha tenido quejas de clientes que compran productos que en realidad ya no tenían stock, y de cupones que no se aplicaron como se esperaba — dos problemas que afectan directamente la relación con el cliente y requieren un plan de pruebas que cubra tanto el sitio público como el panel administrativo.

Alcance funcional del sistema: catálogo de productos con filtros y ordenamiento, ficha de producto con opciones (talla/color, según el producto), carrito de compras, aplicación de cupones/vales de descuento, checkout como invitado o registrado, panel de administración para gestionar productos, stock, categorías y pedidos.

Requisitos funcionales clave:

1. El listado de productos debe soportar ordenamiento por precio y por nombre correctamente.

2. Un producto con opciones (por ejemplo, talla) debe exigir que se seleccione una opción antes de agregarlo al carrito.
3. Cambiar la cantidad de un producto en el carrito debe recalcular el total correctamente.
4. Aplicar un cupón de descuento debe validar el código y aplicar el descuento correcto; un código inválido debe mostrar un error.
5. El checkout debe permitirse completarse como invitado, sin forzar el registro de una cuenta.
6. Al confirmar un pedido, debe generarse un número de orden y un resumen correcto de los productos comprados.
7. Un pedido realizado en el sitio público debe reflejarse correctamente en la lista de pedidos del panel de administración.
8. Al marcar un producto como agotado en el panel de administración, el sitio público debe reflejar esa condición (por ejemplo, impidiendo la compra o mostrando "sin stock").

Requisitos no funcionales: el panel de administración debe reflejar cambios del sitio público en tiempo razonable (sin necesidad de reinicios), el sitio debe funcionar correctamente en los navegadores más usados, tiempo de respuesta del catálogo menor a 2 segundos.

Riesgos iniciales: venta de productos sin stock real, cupones que se aplican incorrectamente o de forma duplicada, inconsistencias entre lo que ve el cliente y lo que administra el equipo de operaciones.

Caso 4 — Reservas de hotel con API (Restful-booker Platform)

Sistema real: Restful-booker Platform — <https://automationintesting.online/>

Empresa y problemática: Una cadena de hoteles boutique digitalizó la gestión de sus habitaciones y reservas: los huéspedes reservan directamente desde el sitio público, y el equipo de recepción administra habitaciones y mensajes desde un panel interno. Además, el sistema expone una API REST que en el futuro usarán agencias de viaje asociadas para consultar disponibilidad. La cadena necesita asegurar que las reservas no se dupliquen sobre las mismas fechas y que lo que ve un huésped en el sitio público coincida exactamente con lo que administra el equipo de recepción — y con lo que devolvería la API a un tercero.

Alcance funcional del sistema: visualización pública de habitaciones disponibles, formulario de reserva (datos de contacto y fechas), formulario de contacto/mensajes, panel de administración (login) para gestionar habitaciones (crear/editar/eliminar) y ver mensajes recibidos, API REST para consultar y crear reservas (endpoint de bookings).

Requisitos funcionales clave:

1. El formulario de reserva debe exigir nombre, correo, teléfono y fechas antes de permitir el envío.
2. La fecha de salida (checkout) no debe poder ser anterior o igual a la fecha de entrada (check-in).
3. El acceso al panel de administración debe requerir login válido.
4. Crear, editar o eliminar una habitación desde el panel debe reflejarse correctamente en el listado público de habitaciones.
5. Un mensaje enviado desde el formulario de contacto público debe aparecer en la bandeja de mensajes del panel de administración.
6. El sistema debe manejar de forma consistente un intento de reservar una habitación en fechas que ya están reservadas (impidiendo la reserva o marcándola explícitamente).
7. Los datos devueltos por la API (por ejemplo, al consultar una reserva) deben coincidir con lo mostrado en la interfaz web.
8. Al cerrar sesión del panel de administración, un intento posterior de acceder a una función administrativa debe exigir volver a iniciar sesión.

Requisitos no funcionales: la API debe responder en menos de 1 segundo ante consultas simples, el panel de administración debe ser accesible solo mediante sesión autenticada, el sitio público debe cargar correctamente sin necesidad de autenticación.

Riesgos iniciales: doble reserva sobre la misma habitación y fechas, inconsistencia entre lo que muestra la API y lo que muestra la interfaz web, mensajes de contacto que no llegan al panel administrativo.

Caso 5 — Gestión de Recursos Humanos (OrangeHRM)

Sistema real: OrangeHRM — <https://opensource-demo.orangehrmlive.com/>

Empresa y problemática: Una empresa mediana implementó OrangeHRM para centralizar la gestión de su personal: registro de empleados, solicitud y aprobación de vacaciones/permisos, reclutamiento de nuevos candidatos y evaluaciones de desempeño. El área de RRHH ha reportado casos de solicitudes de vacaciones aprobadas por error y de empleados duplicados en el sistema — errores que impactan directamente en la planilla y en la relación con los trabajadores, por lo que la empresa exige un plan de pruebas riguroso antes de depender del sistema para decisiones de personal.

Alcance funcional del sistema: gestión de información de empleados (PIM), solicitud y aprobación de permisos/vacaciones (Leave), registro de horas trabajadas (Time), gestión de procesos de

reclutamiento (Recruitment), evaluaciones de desempeño (Performance), administración de usuarios del sistema (Admin).

Requisitos funcionales clave:

1. Agregar un nuevo empleado en el módulo PIM debe exigir al menos nombre y apellido.
2. Una solicitud de permiso/vacaciones debe exigir tipo de permiso y un rango de fechas válido dentro de lo disponible para el empleado.
3. Una solicitud de permiso pendiente debe poder ser aprobada o rechazada por un supervisor, y ese estado debe reflejarse correctamente para el empleado solicitante.
4. Registrar un nuevo candidato en Reclutamiento debe exigir nombre y asociarlo a una vacante existente.
5. El registro de horas (timesheet) debe exigir que se completen entradas por proyecto/tarea antes de poder enviarse.
6. La creación de un usuario del sistema (Admin) debe requerir vincularlo a un registro de empleado ya existente en PIM.
7. Una evaluación de desempeño debe requerir un evaluador asignado y un periodo de evaluación válido.
8. La búsqueda de empleados debe devolver resultados correctos al filtrar por nombre, ID o estado (activo/inactivo).

Requisitos no funcionales: los cambios de estado (por ejemplo, aprobación de un permiso) deben reflejarse de inmediato para todos los roles involucrados, el sistema debe manejar correctamente el acceso simultáneo de varios usuarios (empleado, supervisor, admin), los datos de un empleado eliminado no deben seguir apareciendo en reportes activos.

Riesgos iniciales: aprobación incorrecta de vacaciones (por ejemplo, sin validar el saldo disponible), empleados duplicados en el sistema, desincronización entre el estado que ve el empleado y el que ve su supervisor.

5. Entregables requeridos

El proyecto se organiza en tres entregables, alineados a las actividades del proceso de pruebas ISTQB, presentados como un solo informe con tres secciones diferenciadas.

5.1 Planificación de pruebas

- **Plan de pruebas completo**, usando la plantilla que se les va enviar (alcance, objetivos, criterios de entrada/salida, estimación de esfuerzo, recursos, riesgos, cronograma).
- **Estrategia de pruebas justificada**: tipo(s) de estrategia elegidos y su relación con el contexto del caso (riesgo del negocio, criticidad de los datos, frecuencia de cambios).
- **Niveles de prueba a aplicar**: qué se cubrirá en pruebas de componente, integración, sistema y/o aceptación, y con qué profundidad.

5.2 Análisis de pruebas

- **Identificación de condiciones de prueba** a partir de los requisitos funcionales y no funcionales entregados en el caso (la "base de pruebas").
- **Matriz de trazabilidad** Requisito → Condición(es) de prueba, verificando que ningún requisito clave quede sin cobertura.
- **Priorización de condiciones** según riesgo (alto / medio / bajo), con una breve justificación de cada prioridad alta.

5.3 Diseño de pruebas

- **Casos de prueba** derivados de las condiciones priorizadas como alto y medio riesgo (mínimo 15 casos de prueba en total, cubriendo al menos 4 funcionalidades distintas del sistema real).
- Cada caso de prueba debe incluir: ID, condición de prueba relacionada, precondiciones, pasos, datos de prueba, resultado esperado y prioridad.
- **Técnicas de diseño aplicadas**: aplicar técnicas de caja negra distintas (por ejemplo: partición de equivalencia, análisis de valores límite, tabla de decisión, transición de estados), indicando en qué casos se aplicó cada una y por qué.
- **Especificación de datos de prueba** necesarios para ejecutar los casos diseñados sobre el sistema real.

5.4 Ejecución manual de casos críticos y hallazgos

- **Selección de casos a ejecutar**: todos los casos de prueba diseñados con prioridad **alta**. No se ejecuta el resto del diseño — la selección misma, por prioridad, ya es parte de lo que se evalúa.
- **Ejecución manual** de cada caso seleccionado sobre el sistema real, registrando: resultado esperado, resultado obtenido, y veredicto (pasó / falló / bloqueado).
- **Reporte de hallazgos relevantes**: no tiene que ser necesariamente un defecto — puede ser un comportamiento inesperado, una validación ausente, o una confirmación importante de que algo

crítico sí funciona como debería. Cada hallazgo debe incluir: resumen, pasos para reproducir, resultado esperado vs. obtenido, severidad/prioridad estimada, y evidencia (captura de pantalla).

- **Reflexión de cierre** : de los casos ejecutados manualmente, ¿cuáles recomendarían automatizar en el Proyecto 2 y cuáles no, y por qué? Deben sustentar la respuesta con lo observado durante la ejecución (estabilidad del flujo, frecuencia de repetición esperada, complejidad de la validación), no con la teoría general de cuándo automatizar.

6. Formato de entrega

- Informe en Word o PDF, con todo lo solicitado y claramente delimitadas.
- Los casos de prueba y la matriz de trazabilidad pueden presentarse como tablas dentro del mismo informe o como anexo en Excel referenciado desde el informe.
- Reporte de pruebas con los hallazgos encontrados en la ejecución manual.
- Nombre de archivo sugerido: Proyecto1_Caso[N]_Grupo[X].docx